

UNIVERSITY INSTITUTE OF COMPUTING

MINI PROJECT

ON

ONLINE EXAMINATION

PYTHON PROGRAMMING

Submitted By:

Name: Simran

UID: 24bca10521

Branch: BCA

Group :24BCA3(A)

Semester:5th

Submitted to:

Mr. Shobit

INDEX:

1. Abstract

2. Project Design/Implementation

3. Python Code

4. Output/Screenshots

5. GitHub Repository Link

ABSTRACT

The Online Examination System is a web-based application developed using Python, Flask, and SQLite. The main objective of this project is to make the examination process easier, faster, and more organized by allowing students to take online examinations through a web browser.

The system provides a student interface where students can enter their details and attempt multiple-choice questions. It automatically checks the submitted answers, calculates marks and percentage, and displays the final result with PASS or FAIL status.

The project also includes an Admin Panel that allows the administrator to log in, add examination questions, and view students' examination results. SQLite is used to store examination questions and student results in a database. Flask handles web requests, application routes, and the interaction between the user interface and the database.

The user interface is designed using HTML and CSS to provide a clean, attractive, and user-friendly experience. The system reduces manual checking and helps save time in evaluating objective-type examinations.

The project can be further enhanced by implementing secure student registration, username and password authentication, an examination

timer, and detailed result analytics. Overall, this project demonstrates the practical use of Python programming, web development, database management, and basic authentication in an educational application.

Project Design/Implementation

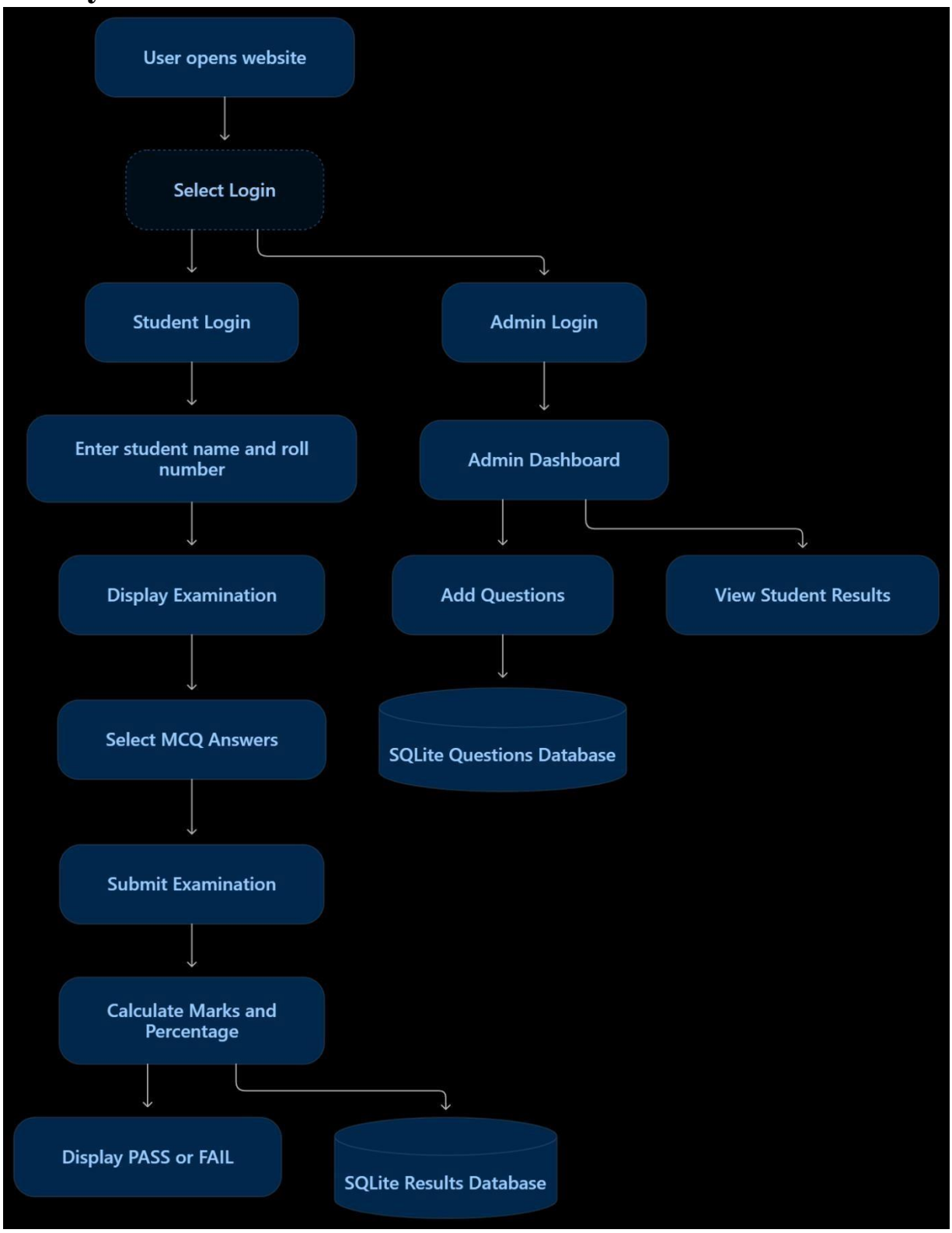
✞ Introduction

The Online Examination System is developed as a web application using Python Flask for backend processing, HTML and CSS for the frontend, and SQLite for database management.

✞ Technologies used

1. Python: Application logic and result calculation
2. Flask: Web framework and URL routing
3. HTML: Structure of web pages and examination forms
4. CSS: Website design and responsive layout
5. SQLite: Storing questions and examination results
6. Jinja2: Displaying dynamic content in HTML templates
7. VS Code: Writing and running the code
8. Web browser: Accessing the examination system

✞ System architecture



✞ Main modules

Module 1: Student Login

Students enter their name and roll number to access the examination. Secure username/password registration is a planned upgrade.

Module 2: Examination

The application displays multiple-choice questions and four answer options for each question. Students select answers and submit the form.

Module 3: Automatic Evaluation

The system compares submitted answers with the correct answers and calculates the total score, percentage, and PASS/FAIL status. The current passing threshold is 40%.

Module 4: Admin Panel

The administrator can log in, add new questions, and view examination results.

Module 5: Database Management

SQLite maintains the questions and results tables. A students table has also been introduced for the planned registration feature; the registration and password-login flow still needs to be implemented and tested.

✞ Database design

The database uses the following tables:

Table 1: questions

Field	Description
id	Unique question ID
question	Question text
option1–option4	Four answer options
answer	Index of the correct option (0–3)

Table 2: results

Field	Description
id	Unique result ID
student_name	Student's name
roll_number	Student's roll number
score	Marks obtained
total	Total questions
percentage	Percentage obtained
status	PASS or FAIL

Table 3: students — planned authentication

Field	Description
id	Unique student ID
student_name	Registered name
roll_number	Unique roll number
username	Unique login username
password_hash	Hashed password

✞ Implementation procedure:

1. Install Python and Flask
2. Create the project file app.py.
3. Initialize the SQLite database and required tables.
4. Design the login page and examination interface using HTML and CSS.
5. Create Flask routes for student login, examination, and submission.
6. Implement automatic answer checking and result calculation.
7. Create the admin login and dashboard.
8. Implement the question-adding functionality.
9. Test the application and verify that questions and results are stored correctly.

✞ Result calculation

The system calculates the percentage using the formula:

Percentage= marks obtained/ total questions*100

The current program assigns the result as follows:

Percentage of 40% or above: PASS
Percentage below 40%: FAIL

Python Code:

```
from flask import Flask, request, redirect, session, render_template_string import sqlite3

app = Flask(__name__)  app.secret_key =
"exam_project_key_change_later"

DATABASE = "exam.db"

# ---------- CREATE DATABASE ---------- def
create_database():
   conn = sqlite3.connect(DATABASE)

   conn.execute("""
      CREATE TABLE IF NOT EXISTS results (            id
INTEGER PRIMARY KEY AUTOINCREMENT,
         student_name TEXT NOT NULL,
roll_number             TEXT       NOT
               NULL,           score INTEGER NOT
NULL,           total INTEGER NOT NULL,
percentage REAL NOT NULL,
         status TEXT NOT NULL
      )
   """)


   conn.execute("""
      CREATE TABLE IF NOT EXISTS questions (            id
INTEGER PRIMARY KEY AUTOINCREMENT,
         question TEXT NOT NULL,
option1 TEXT NOT NULL,           option2
TEXT NOT NULL,           option3 TEXT
NOT NULL,           option4 TEXT NOT
NULL,           answer INTEGER NOT
NULL       )
   """)

   conn.commit()
conn.close()

create_database()
```

```
# ---------- EXAM QUESTIONS ---------- questions
= [
    {
        "question": "1. What is the correct file extension for Python?",
        "options": [".html", ".py", ".java", ".cpp"],
        "answer": 1
    },
    {
        "question": "2. Which keyword is used to define a function in Python?",
        "options": ["func", "define", "def", "function"],
        "answer": 2
    },
    {
        "question": "3. Which data type stores True or False?",
        "options": ["int", "str", "list", "bool"],
        "answer": 3
    },
    {
        "question": "4. Which symbol is used for comments in Python?",
        "options": ["//", "#", "/*", "&"],
        "answer": 1
    },
    {
        "question": "5. Which function displays output in Python?",
        "options": ["input()", "show()", "print()", "display()"],
        "answer": 2
    }
]
# ---------- LOAD ALL EXAM QUESTIONS ---------- def
get_exam_questions():     # Start with the original five questions
    all_questions = list(questions)

    # Read admin-added questions from the database
conn = sqlite3.connect(DATABASE)
conn.row_factory = sqlite3.Row

    rows = conn.execute("""
        SELECT question, option1, option2, option3, option4, answer
        FROM questions
        ORDER BY id
    """).fetchall()

    conn.close()

    # Avoid adding a question twice if it already exists
    # in the original list          existing_questions = {
q["question"].strip().lower().lstrip("0123456789.        )")
for q in all_questions
   }

    for row in rows:
        question_text = row["question"].strip()         normalized =
question_text.lower().lstrip("0123456789. )")
```

```
        if normalized not in existing_questions:
            all_questions.append({
"question": question_text,
"options":                       [
row["option1"],
row["option2"],
row["option3"],
                    row["option4"]
                ],
                "answer": row["answer"]
            })

            existing_questions.add(normalized)

    return all_questions



# ---------- CSS DESIGN ----------

# ---------- MODERN CSS DESIGN ---------- STYLE
= """
<style>
* {      box-sizing: border-
box;
}

body {     margin: 0;      font-family: 'Segoe UI', Arial,
sans-serif;     background: linear-gradient(135deg, #eef2ff,
#f8faff, #f3e8ff);     color:
#1e293b;     min-height: 100vh;
padding: 28px 16px;
}

.container {     background: white;     width: 100%;
max-width: 760px;    margin: 24px auto;    padding:
30px;    border-radius: 22px;    box-shadow: 0 12px
40px rgba(54, 65, 120, 0.12);     text-align: left;
}

h1, h2 {      color:
#26356c;
}

h1  {      text-align:
center;   font-size:
30px; } h2 {    text-
align:
center;
}

input[type=text],
input[type=password], select
{     width: 100%;     padding:
13px 14px;     margin: 8px 0
```

```
18px;     border: 1px solid
#d8def0;     border-radius:
10px;     background: #fbfcff;
fontsize: 15px;     outline:
none;
}

input:focus, select:focus {     bordercolor:
#6558e8;     box-shadow: 0 0 0 3px rgba(101, 88,
232, 0.12);
}

button {     background: linear-gradient(135deg, #4f46e5,
#7c3aed);     color: white;     padding: 13px 22px;     border:
none;     borderradius: 10px;     cursor: pointer;     font-
size: 15px;     font-weight: 600;     transition: transform
0.2s, box-shadow 0.2s;
}

button:hover {     transform: translateY(-2px);     box-shadow: 0
7px 18px rgba(79, 70, 229, 0.25);
}

a {     color: #5145cd;     text-decoration:
none;
}

a:hover {     text-decoration:
underline;
}

.question {     padding: 16px 0;     border-bottom: 1px
solid #e7eaf3;
}

.option {     display:
block;     margin: 12px
0;     cursor: pointer; }
.result {     textalign:
center;
}

.brand {     text-align:
center;     font-size:
14px;     font-weight:
700;     letter-spacing:
3px;     color:
#6255df;
}

.hero {     text-align:
center;     padding:
20px 0 30px;
}

.hero h1 {     font-size: 38px;     margin: 12px 0;
background: linear-gradient(135deg, #3730a3, #9333ea);
```

```
-webkit-background-clip: text;     backgroundclip: text;
color: transparent;
}

.hero p {     color:
#64748b;     lineheight:
1.7;
}

.login-grid {     display: grid;      grid-template-
columns: repeat(2, minmax(0, 1fr));      gap: 22px;
}

.login-card {     padding:
24px;     border: 1px solid
#e5e7f5;     border-radius: 18px;      background: linear-
gradient(180deg, #ffffff, #fafaff);     boxshadow: 0 8px 22px
rgba(49, 46, 129, 0.07);
}

.login-card h2 {
fontsize: 22px;
margin: 12px 0;
}

.card-icon {    width: 55px;
height: 55px;    border-
radius:      16px;
background: #eeedff;
   color: #5145cd;
  display: flex;
   align-items: center; justifycontent:
   center; font-size: 27px;    margin: 0
   auto;
}

.login-card                p
              {     color:
              #64748b;     font-
size:         14px;     line-
height:      1.6;     text-align:
center;
}

.login-card button {     width:
              100%;
margin-top: 5px;
}

.footer {     textalign:
center;     color:
#94a3b8;     font-size:
12px;     margin-top:
25px;
}
```

```
@media (max-width: 600px) {
  body {        padding:
14px 10px;
  }

  .container                {
padding:              20px;
margin: 10px auto;
  }

  .login-grid {       grid-templatecolumns:
1fr;
  }
  .hero h1 {       font-size:
29px;
  }
}
</style>
"""

# ---------- HOME / STUDENT LOGIN ----------

# ---------- HOME PAGE ----------
@app.route("/") def home():     return
render_template_string("""
 <!DOCTYPE html>
 <html>
<head>
    <title>ExamPortal | Online Examination</title>
 """ + STYLE + """
 </head>
 <body>
   <div class="container">

     <div class="brand">EXAMPORTAL</div>
     <div class="hero">
       <h1>Your Journey Starts Here</h1>
       <p>
         Welcome to your online examination platform.
Learn, attempt exams, and check your results              with
a simple and organized experience.
       </p>
     </div>

      <div class="login-grid">

        <!-- STUDENT LOGIN CARD -->
        <div class="login-card">
          <div class="card-icon">&#128100;</div>

          <h2>Student Login</h2>

          <p>
```

```
                Enter your details to begin your
examination.
            </p>
            <form action="/login" method="POST">
              <label>Student Name</label>
              <input                    type="text"
name="student_name"
                placeholder="Enter your full name"
required>
              <label>Roll Number</label>
              <input                    type="text"
name="roll_number"
                placeholder="Enter your roll number"
required>
              <button type="submit">
                Start Examination &#8594;
              </button>
            </form>
          </div>
          <!-- ADMIN LOGIN CARD -->
          <div class="login-card">
            <div class="card-icon">&#128274;</div>

            <h2>Admin Login</h2>

            <p>
              Access the admin dashboard to manage                exam
questions and view student results.
            </p>

            <form action="/admin" method="POST">
              <label>Admin Username</label>
              <input            type="text"
                name="username"
                placeholder="Enter admin username"
required>

              <label>Password</label>
              <input                    type="password"
name="password"
placeholder="Enter admin password"                    required>

              <button type="submit">
                Open Admin Portal &#8594;
              </button>
            </form>
          </div>
        </div>

        <div class="footer">
          Online Examination System | Powered by Python Flask
        </div>

      </div>
    </body>
```

```
    </html>
    """)


# ---------- PROCESS LOGIN - ---------- @app.route("/login", methods=["POST"]) def login():     name = request.form.get("student_name", "").strip()
    roll = request.form.get("roll_number", "").strip()

    if not name or not roll:
        return "Please enter your name and roll number.", 400

    session["student_name"] = name
session["roll_number"] = roll

    return redirect("/exam")

# ---------- EXAM PAGE ---------- @app.route("/exam") def exam():      if "student_name" not in session:
return redirect("/")      exam_questions = get_exam_questions()

    return render_template_string("""
  <!DOCTYPE html>
  <html>
  <head>
     <title>Python Examination</title>
  """ + STYLE + """
  </head>
  <body>
     <div class="container">
        <h1>Python Examination</h1>          <p><strong>Student:</strong> {{ name }}</p>
        <p><strong>Roll Number:</strong> {{ roll }}</p>
        <p><strong>Total Questions:</strong> {{ questions|length }}</p>

        <form action="/submit" method="POST">
           {% for q in questions %}
              {% set question_number = loop.index0 %}

              <div class="question">
                 <h3>{{ q.question }}</h3>

                 {% for option in q.options %}
                    <label class="option">
                       <input                          type="radio" name="q{{ question_number }}" value="{{ loop.index0 }}"                          required>
                       {{ option }}
                    </label>
                 {% endfor %}
              </div>
           {% endfor %}
```

```
            <p style="text-align:center; margin-top:25px">
              <button type="submit">
                Submit Examination
              </button>
            </p>
          </form>
        </div>
    </body>
    </html>
    """,
questions=exam_questions,     name=session["student_name"],
roll=session["roll_number"])

# ---------- SUBMIT EXAM AND SAVE RESULT ----------
@app.route("/submit",  methods=["POST"])  def  submit():        if
"student_name" not in session:
        return redirect("/")

    score = 0     exam_questions = get_exam_questions()

    # Check each submitted answer    for i, q in
enumerate(exam_questions):        selected =
request.form.get("q" + str(i))

      print("Question:", i + 1)       print("Selected
answer:", selected)      print("Correct answer:",
q["answer"])     if selected is not None and
selected.isdigit():
          if int(selected) == q["answer"]: score
            += 1
  total = len(exam_questions)    percentage = (score
/ total) * 100    status = "PASS" if percentage >= 40
else "FAIL"    print("FINAL SCORE:", score)

  # Save result to SQLite database    conn
= sqlite3.connect(DATABASE)

    conn.execute("""         INSERT
INTO results
        (student_name, roll_number, score, total, percentage, status)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (
session["student_name"],
        session["roll_number"],        score,
        total,
percentage,        status
    ))

    conn.commit()
conn.close()

    # Display result page     return render_template_string("""
    <!DOCTYPE html>
```

```
  <html>
  <head>
    <title>Examination Result</title>
  """ + STYLE + """
  </head>
  <body>
    <div class="container result">
      <h1>Examination Result</h1>

      <h3>Student: {{ name }}</h3>
      <p><strong>Roll Number:</strong> {{ roll }}</p>

      <h2>Marks: {{ score }} / {{ total }}</h2>

      <h3>Percentage: {{ "%.2f"|format(percentage) }}%</h3>

      <h2>Result: {{ status }}</h2>

      <p><a href="/">Return to Login</a></p>
    </div>
  </body>
  </html>
  """,
name=session["student_name"],
roll=session["roll_number"],
score=score,    total=total,
percentage=percentage,
status=status)

# ---------- RUN APPLICATION ---------- @app.route("/admin", methods=["GET",
"POST"]) def admin():    if request.method == "POST":
    username = request.form.get("username")
password = request.form.get("password")

    if username == "admin" and password == "admin123":
      session["is_admin"]    =    True
return    redirect("/admin/results")    else:
      return "Invalid username or password. Go back and try again."

  return render_template_string("""
  <html>
  <head>
    <title>Admin Login</title>
  </head>
  <body style="font-family: Arial; text-align: center; padding: 50px;">
    <h1>Admin Login</h1>
    <form method="POST">
      <p>
        <input type="text" name="username"
placeholder="Username" required>
      </p>
      <p>
        <input type="password" name="password"
placeholder="Password" required>
      </p>
```

```
        <button type="submit">Login</button>
      </form>
    </body>
    </html>
    """)

@app.route("/admin/results")
def admin_results():          if not
session.get("is_admin"):
return redirect("/admin")

    conn = sqlite3.connect(DATABASE)
conn.row_factory     =               sqlite3.Row
rows = conn.execute(
      "SELECT * FROM results ORDER BY id DESC"
    ).fetchall()
conn.close()

    return render_template_string("""
    <!DOCTYPE html>
    <html>
    <head>
      <title>Admin Dashboard</title>
      <style>            body
{                font-family:
Arial, sans-serif;
background: #eef2ff;
padding: 25px;         }
          .container {
             background: white;
            padding: 20px;
           border-radius: 12px;               overflow-x:
auto;
         }
         h1 { color: #243b64; text-align: center; }
table {               width: 100%;               bordercollapse:
collapse;
             margin-top: 20px;
         }            th, td {
border: 1px solid #ddd;
padding: 10px;                textalign:
center;
         }
         th { background: #2457a7; color: white; }            tr:nth-
child(even) { background: #f2f5ff; }            a
{ color: #2457a7; }
      </style>
    </head>
    <body>
      <div class="container">
        <h1>Admin Dashboard</h1>
        <h3>Total Exam Attempts: {{ rows|length }}</h3>

        {% if rows %}
        <table>
```

```
            <tr>
              <th>ID</th>
              <th>Student Name</th>
              <th>Roll Number</th>
              <th>Marks</th>
              <th>Percentage</th>
              <th>Status</th>
            </tr>
            {% for r in rows %}
            <tr>
              <td>{{ r["id"] }}</td>
              <td>{{ r["student_name"] }}</td>
              <td>{{ r["roll_number"] }}</td>
              <td>{{ r["score"] }} / {{ r["total"] }}</td>
              <td>{{ "%.2f"|format(r["percentage"]) }}%</td>
              <td>{{ r["status"] }}</td>
            </tr>
            {% endfor %}
          </table>
          {% else %}
            <p>No examination results found yet.</p>
          {% endif %}
          <p><a href="/admin/add-question">Add New Question</a></p>
<p><a href="/admin">Back to Admin Login</a></p>
          <p><a href="/">Student Login Page</a></p>
      </div>
    </body>
    </html>
    """, rows=rows)

@app.route("/admin/add-question", methods=["GET", "POST"])
def add_question(): if not session.get("is_admin"): return
redirect("/admin")

message = ""

  if request.method == "POST":
      question = request.form.get("question")
option1 = request.form.get("option1")       option2
= request.form.get("option2")        option3 =
request.form.get("option3")        option4 =
request.form.get("option4")         answer =
request.form.get("answer", type=int)

      conn = sqlite3.connect(DATABASE)
conn.execute("""             INSERT INTO
questions
          (question, option1, option2, option3, option4, answer)
          VALUES (?, ?, ?, ?, ?, ?)
      """, (question, option1, option2, option3,
option4, answer))        conn.commit()
      conn.close()

      message = "Question added successfully!"
```

```
    return render_template_string("""
    <html>
    <head>
        <title>Add Question</title>
    </head>
    <body style="font-family: Arial; max-width: 600px;
margin: 40px auto; padding: 20px;">        <h1>Add
New Question</h1>
        <p style="color: green;">{{ message }}</p>

        <form method="POST">
            <p>Question</p>
            <input name="question" required style="width: 100%;">

            <p>Option 1</p>
            <input name="option1" required style="width: 100%;">

            <p>Option 2</p>
            <input name="option2" required style="width: 100%;">

            <p>Option 3</p>
            <input name="option3" required style="width: 100%;">
<p>Option 4</p>
            <input name="option4" required style="width: 100%;">

            <p>Correct Answer</p>
            <select name="answer" required style="width: 100%;">
                <option value="0">Option 1</option>
                <option value="1">Option 2</option>
                <option value="2">Option 3</option>
                <option value="3">Option 4</option>
            </select>

            <br><br>
            <button type="submit">Save Question</button>
        </form>

        <p><a href="/admin/results">Back to Dashboard</a></p>
    </body>
    </html>
    """, message=message) if __name__


== "__main__":


    app.run(debug=True)
```

Output/Screenshots

Screenshot 1: Home Page

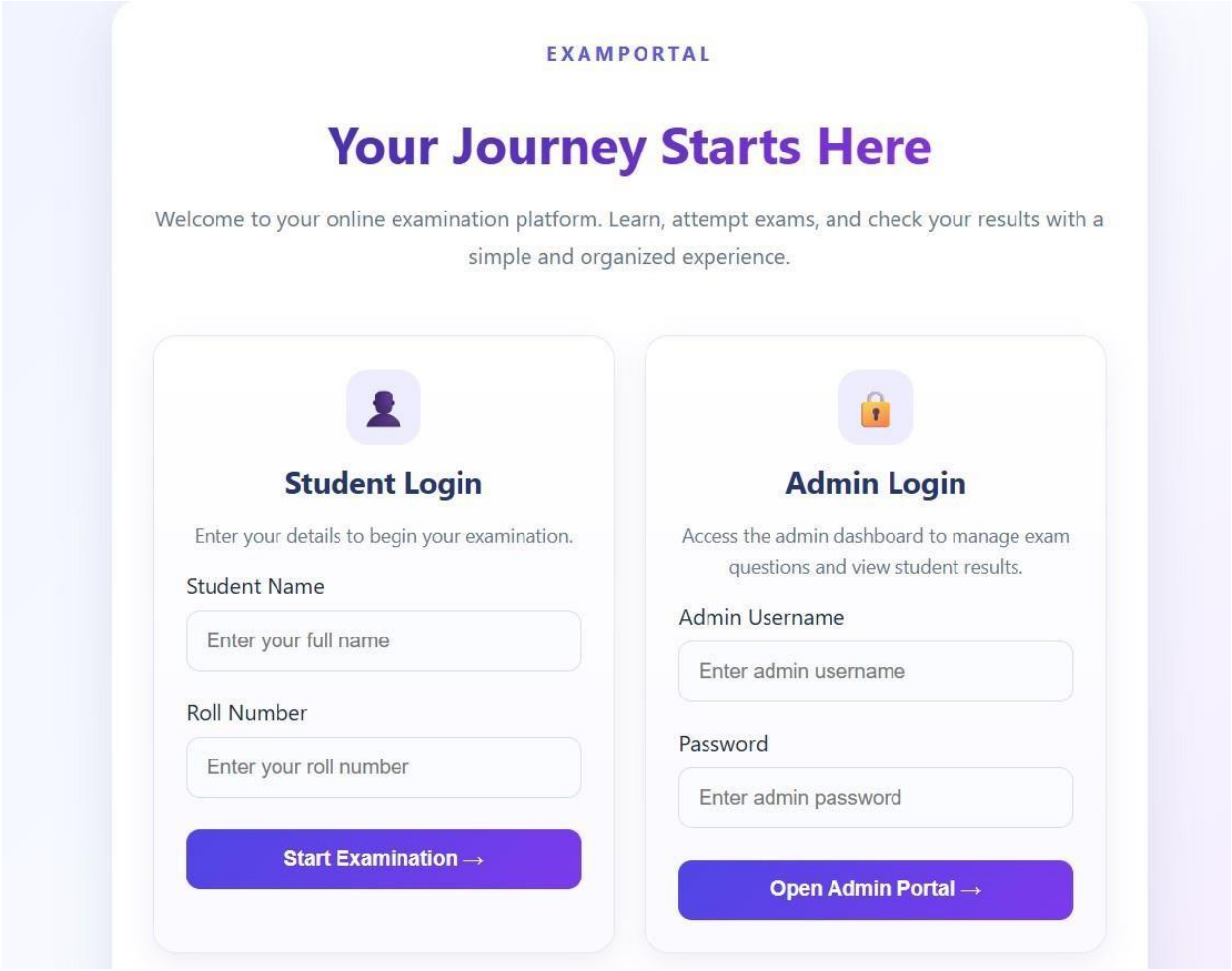
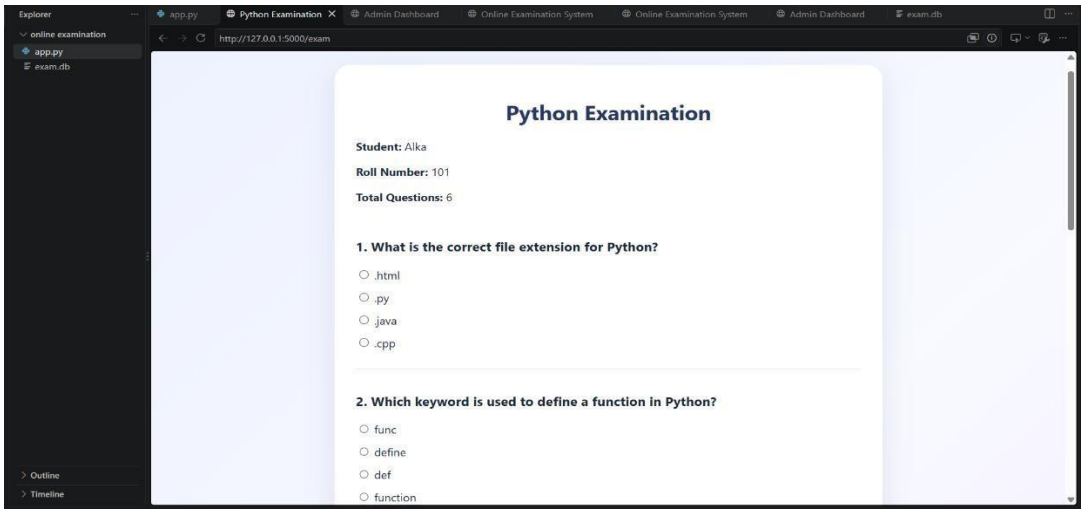


Figure 1 — Home Page of the Online Examination System.

Screenshot 2: Student Examination Page



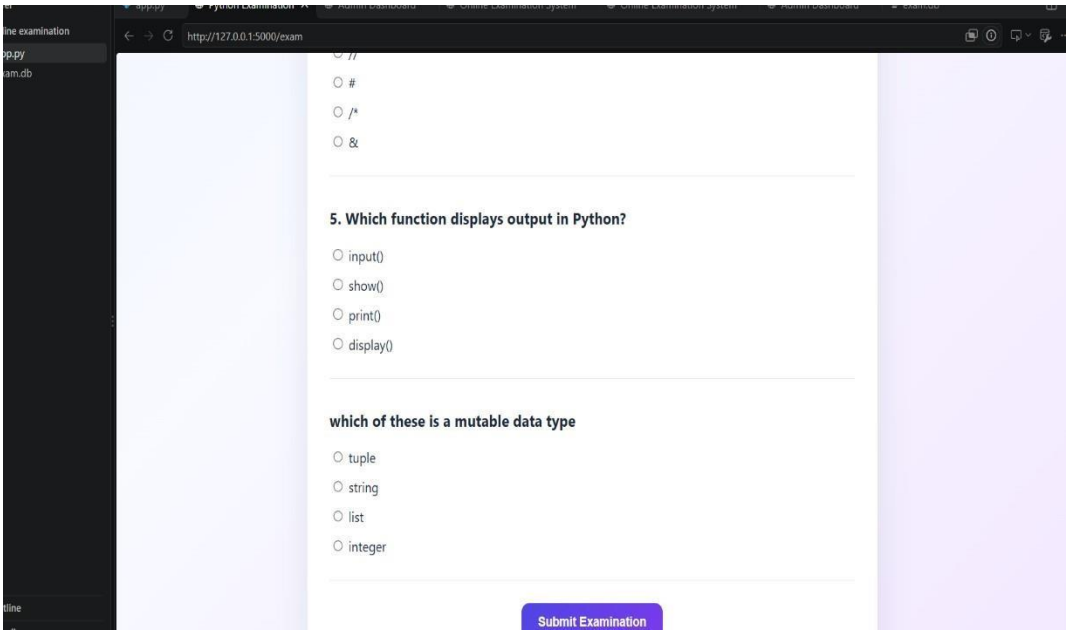


Figure 2 — Online Examination Interface.

Screenshot 3: Examination Result

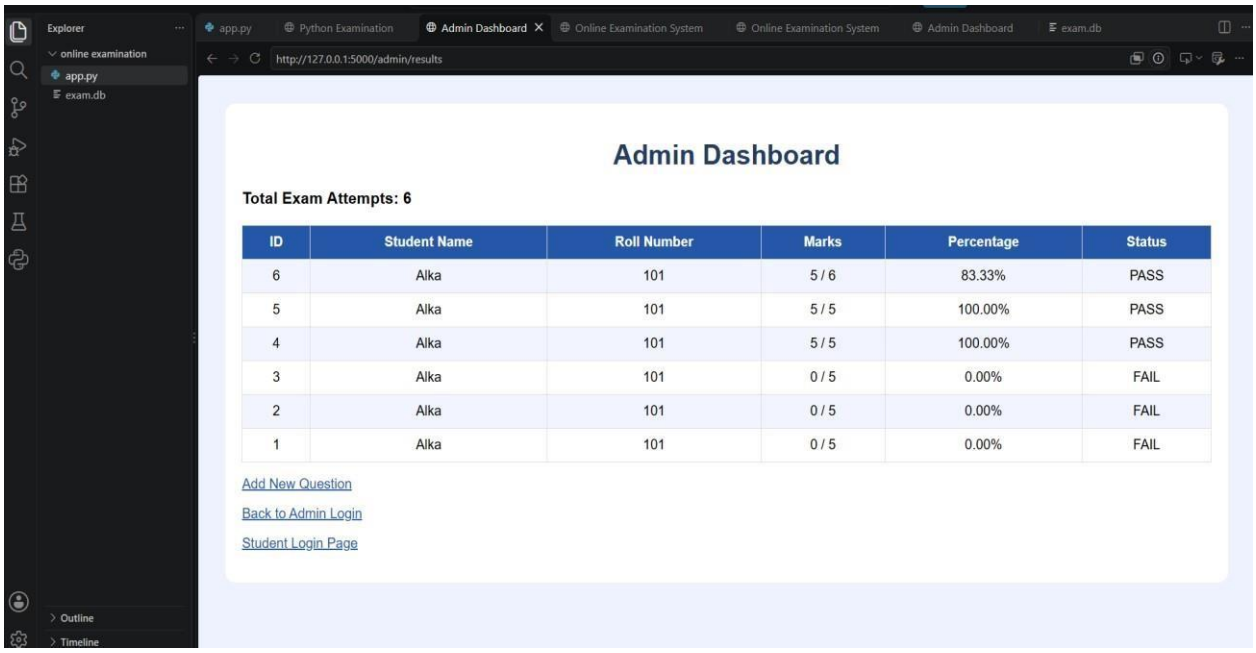


Figure 3 — Automatic Result Generation.

Screenshot 4: Admin Dashboard

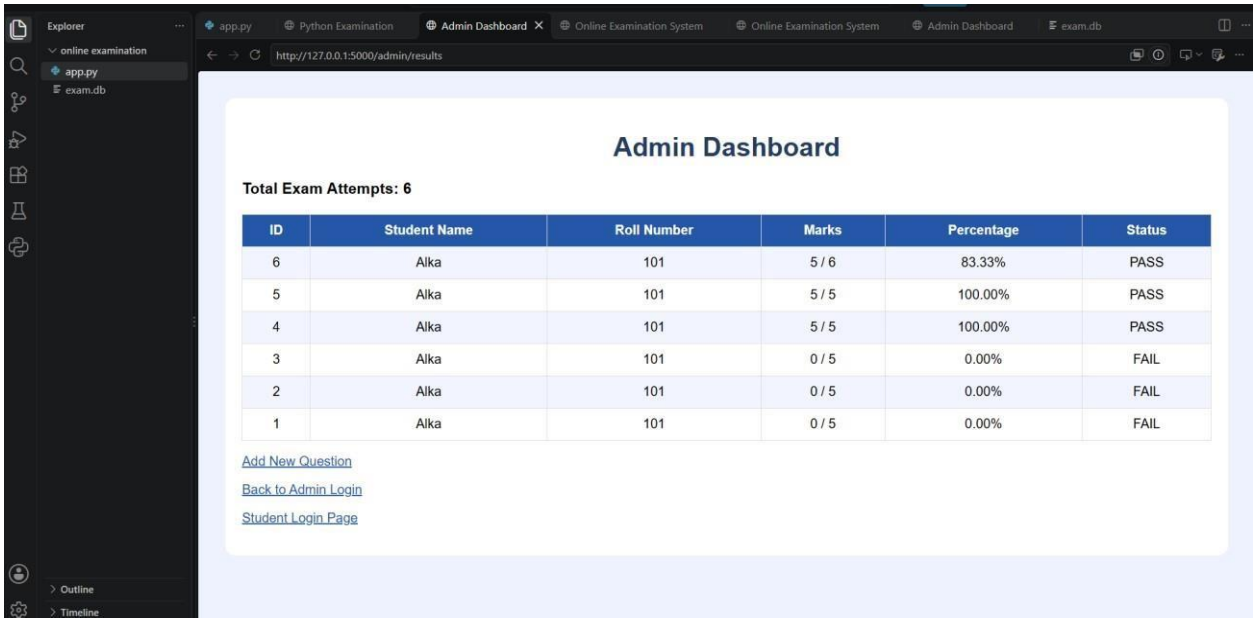
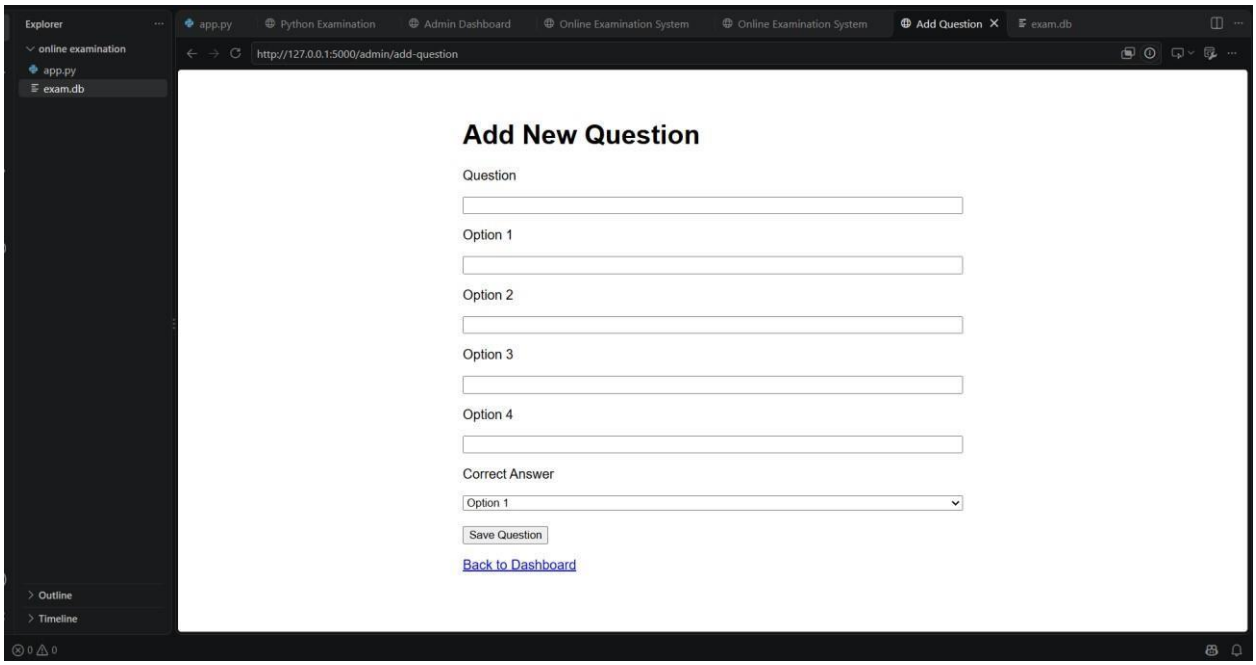


Figure 4 — Administrator Dashboard.

Screenshot 5: Add Question



Question Management Module.

Github link: https://github.com/Alka-Ahlawat/Onlineexamination-system